

이상치 탐지

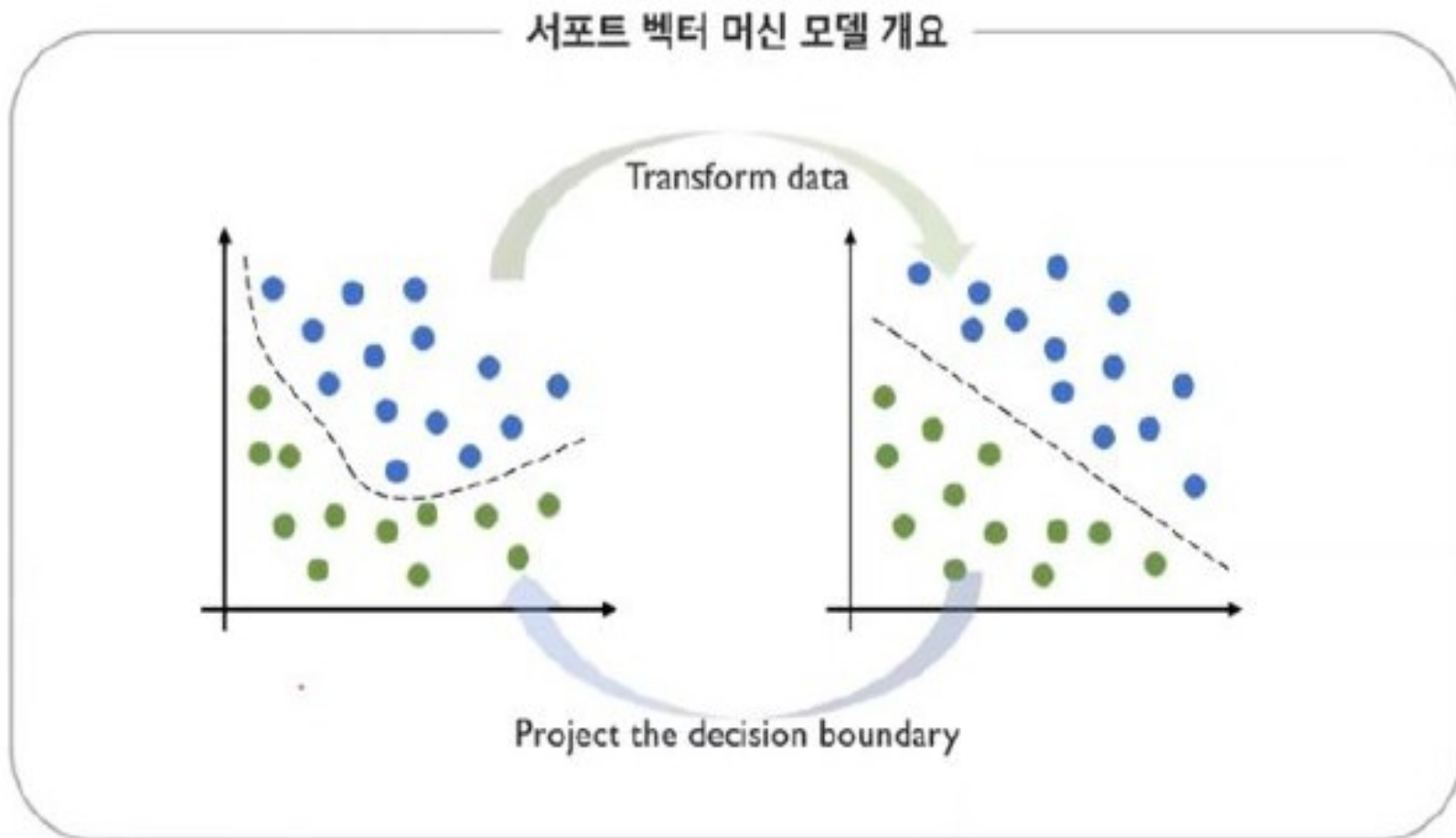
이상치 탐지

- 이상치 탐지 방법 (비지도 학습을 중심으로)
 1. 도메인 기반 EDA
 2. 일변량 방법 (터키 방법, z-점수 등)
 3. 다변량 방법 (MCD, OCSVM, Isolation Forest, LOF 등)

One Class SVM

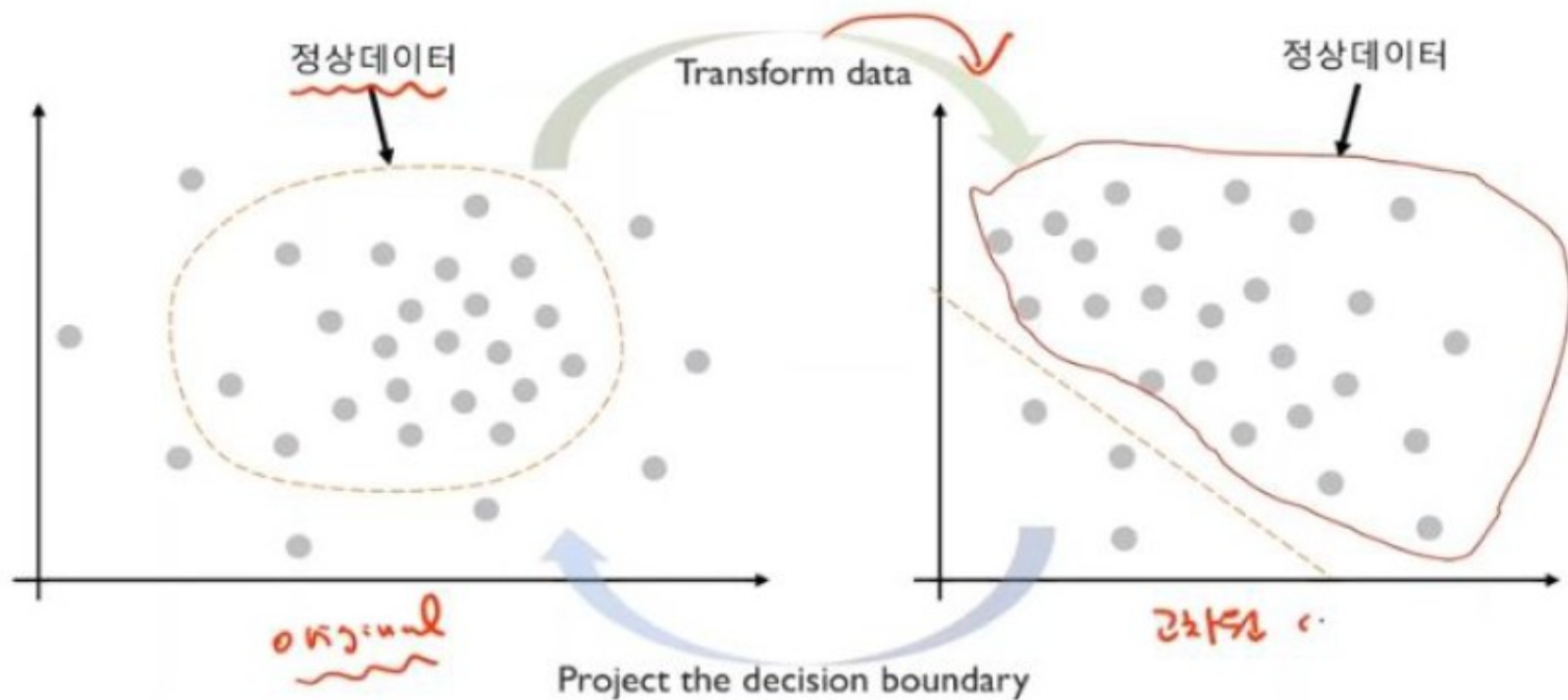
One-Class 서포트 벡터 머신(OC-SVM)

- 정상 관측치와 불량 관측치를 구별하는 서포트 벡터 머신 알고리즘



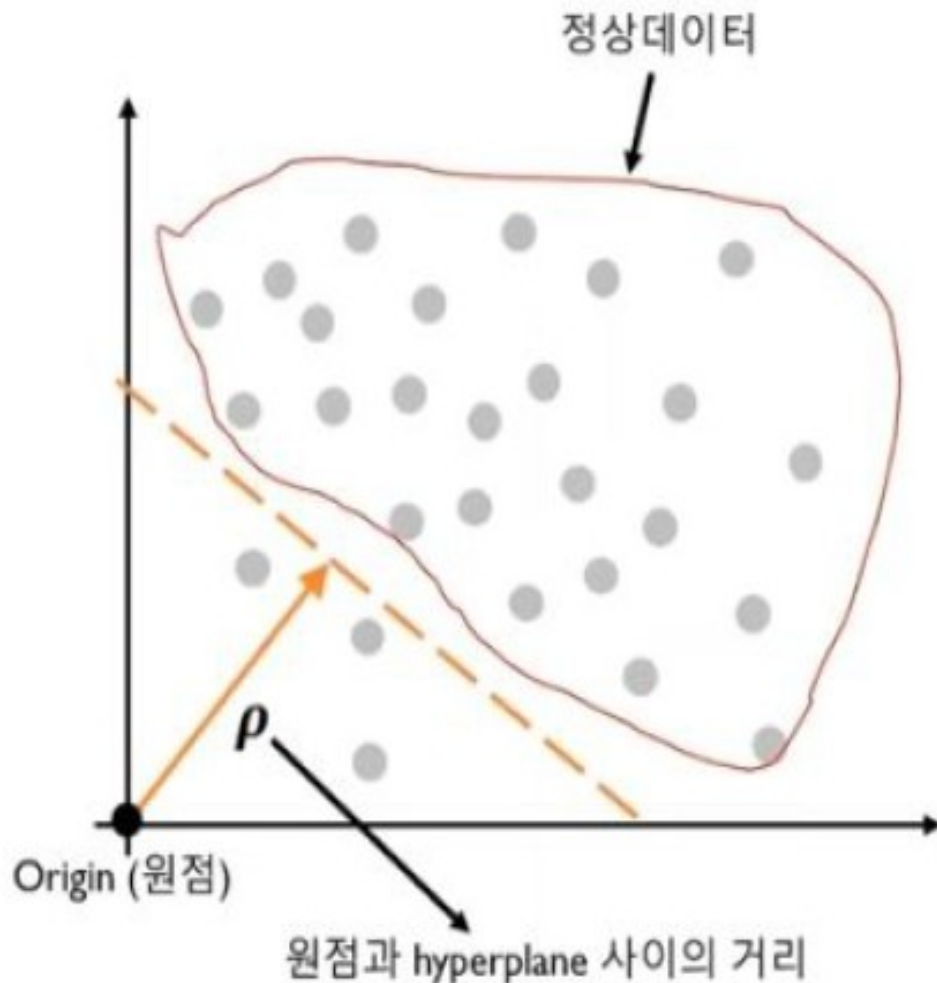
One-Class 서포트 벡터 머신(OC SVM)

- 정상 데이터가 최대한 원점에서 멀리 떨어지도록 의사결정 경계를 구축



One-Class 서포트 벡터 머신(OC SVM)

- 정상 데이터가 최대한 원점에 멀리 떨어지도록 초평면 구축
- ρ 워점에서 의사결정 경계면까지의 거리



$$\min_w \frac{1}{2} \|w\|^2 + \frac{1}{vn} \sum_{i=1}^n \xi_i - \rho$$

$$s.t. \ w \cdot \phi(x_i) \geq \rho - \xi_i$$

$$i = 1, 2, \dots, n, \quad \xi_i \geq 0$$

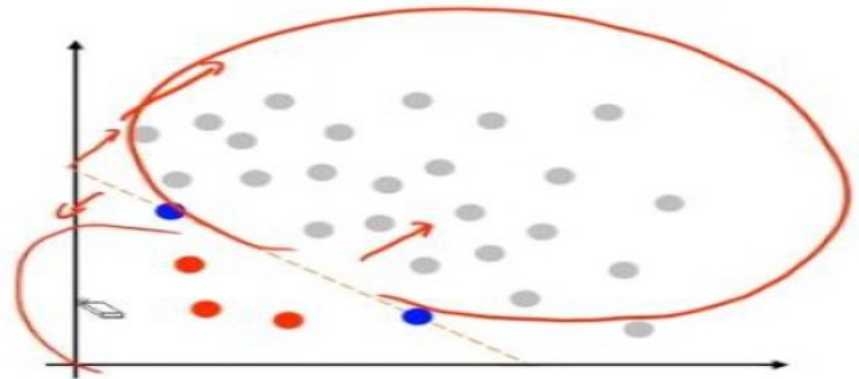
$$f(x_i) = \text{sign}(w \cdot \phi(x_i) - \rho)$$

One-Class 서포트 벡터 머신(OC SVM)

- α 를 기준으로 관측치가 초평면 안/밖/상에 있음

$$\alpha_i = \frac{1}{vn} - \beta_i \quad \sum_{i=1}^n \alpha_i = 1, \quad 0 \leq \alpha_i \leq \frac{1}{vn} \quad \beta_i \xi_i = 0$$

- $\alpha_i = 0$
관측치가 hypersphere 안에 있음
- $\alpha_i = \frac{1}{vn} \Rightarrow \beta_i = 0 \Rightarrow \xi_i > 0$
관측치가 hypersphere 밖에 있음
(support vector)
- $0 < \alpha_i < \frac{1}{vn} \Rightarrow \beta_i > 0 \Rightarrow \xi_i = 0$
관측치가 hypersphere 상에 있음
(support vector)

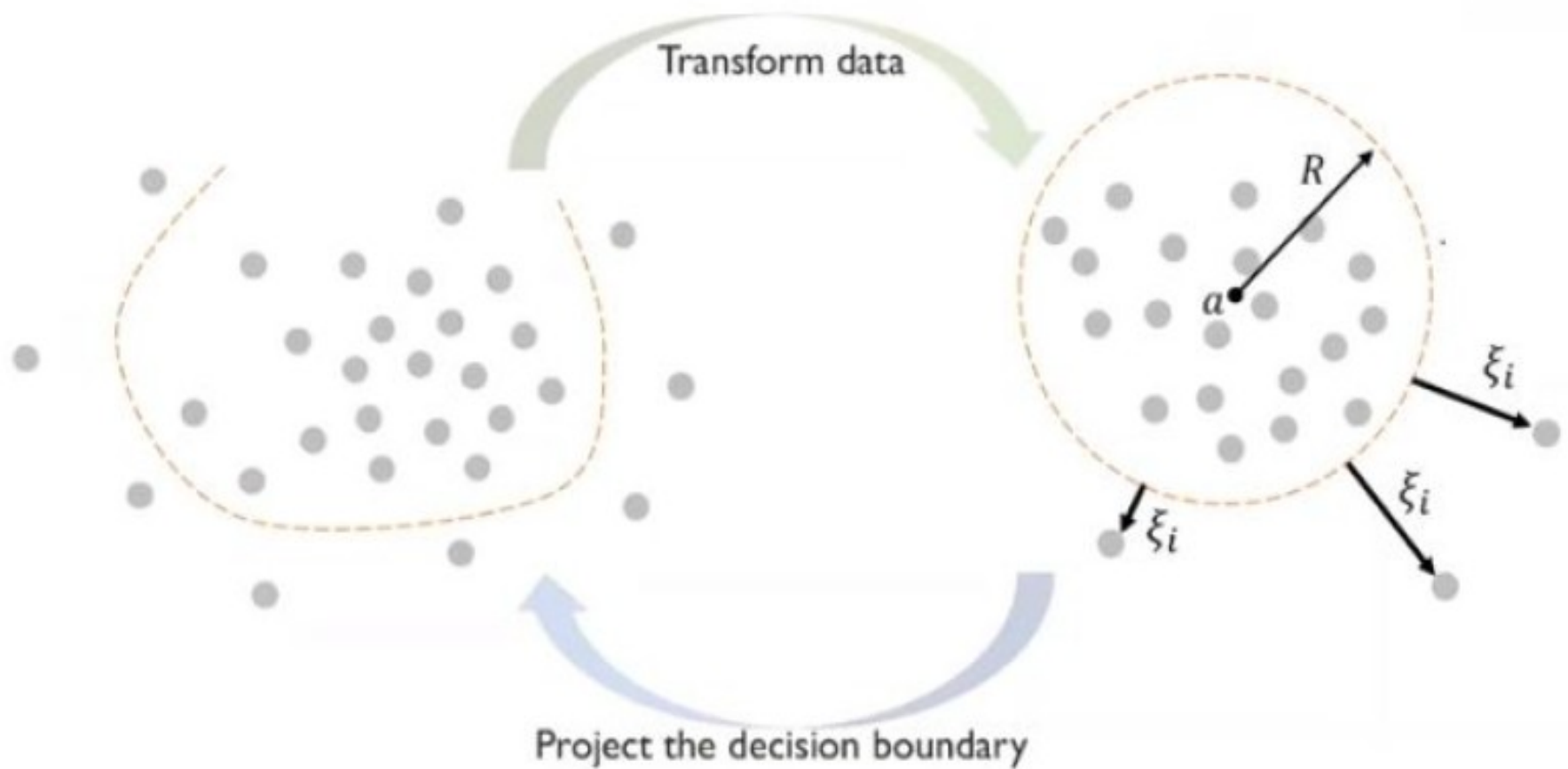


Examples

```
>>> from sklearn.svm import OneClassSVM
>>> X = [[0], [0.44], [0.45], [0.46], [1]]
>>> clf = OneClassSVM(gamma='auto').fit(X)
>>> clf.predict(X)
array([-1,  1,  1,  1, -1])
>>> clf.score_samples(X)
array([1.7798..., 2.0547..., 2.0556..., 2.0561..., 1.7332...])
```

SVDD(Suport Vector Data Description)

- 데이터를 특성공간으로 변환해 매핑
- 정상 관측치를 감싸는 가장 작은 초구를 구축 (중심 a , 반지름 R)



SVDD(Suport Vector Data Description)

- SVDD 공식

반지름을 최소화하여 hypersphere의 크기를 최소화

$$\min_{R, a, \xi_i} R^2 + C \sum_{i=1}^n \xi_i$$

Hypersphere 밖으로 나가는 점들의 거리를 최소화

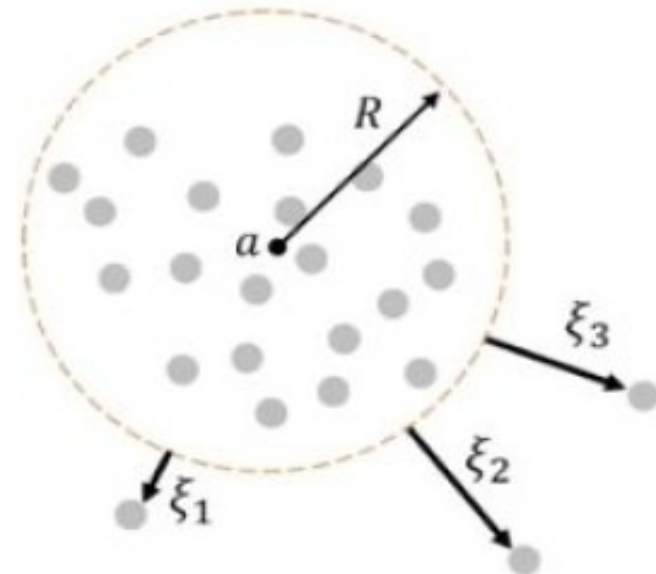
$$s. t. \quad \|\phi(x_i) - a\|^2 \leq R^2 + \xi_i$$

$$\xi_i \geq 0, i = 1, 2, \dots, n.$$

$\phi(\cdot)$: Kernel mapping function

C : Regularization parameter

n : 정상데이터 개수



SVDD(Suport Vector Data Description)

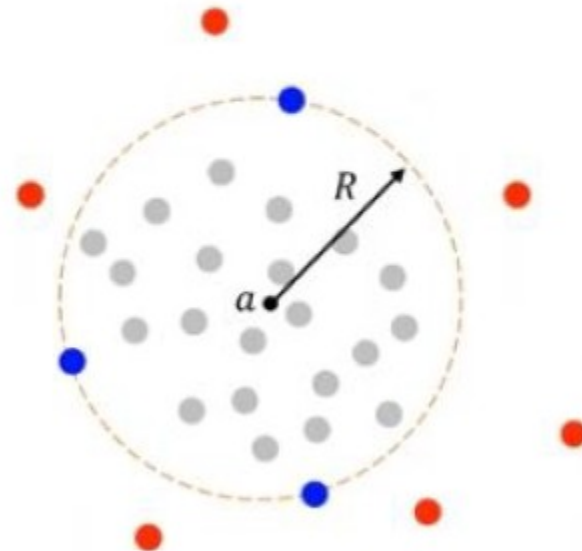
- OCSVM과 같이 α 를 기준으로 관측치가 초평면 안/밖/상에 있음

$$C - \alpha_i - \beta_i = 0 \quad \beta_i \xi_i = 0 \quad 0 \leq \alpha_i \leq C$$

① ● $\alpha_i = 0$
관측치가 hypersphere 안에 있음

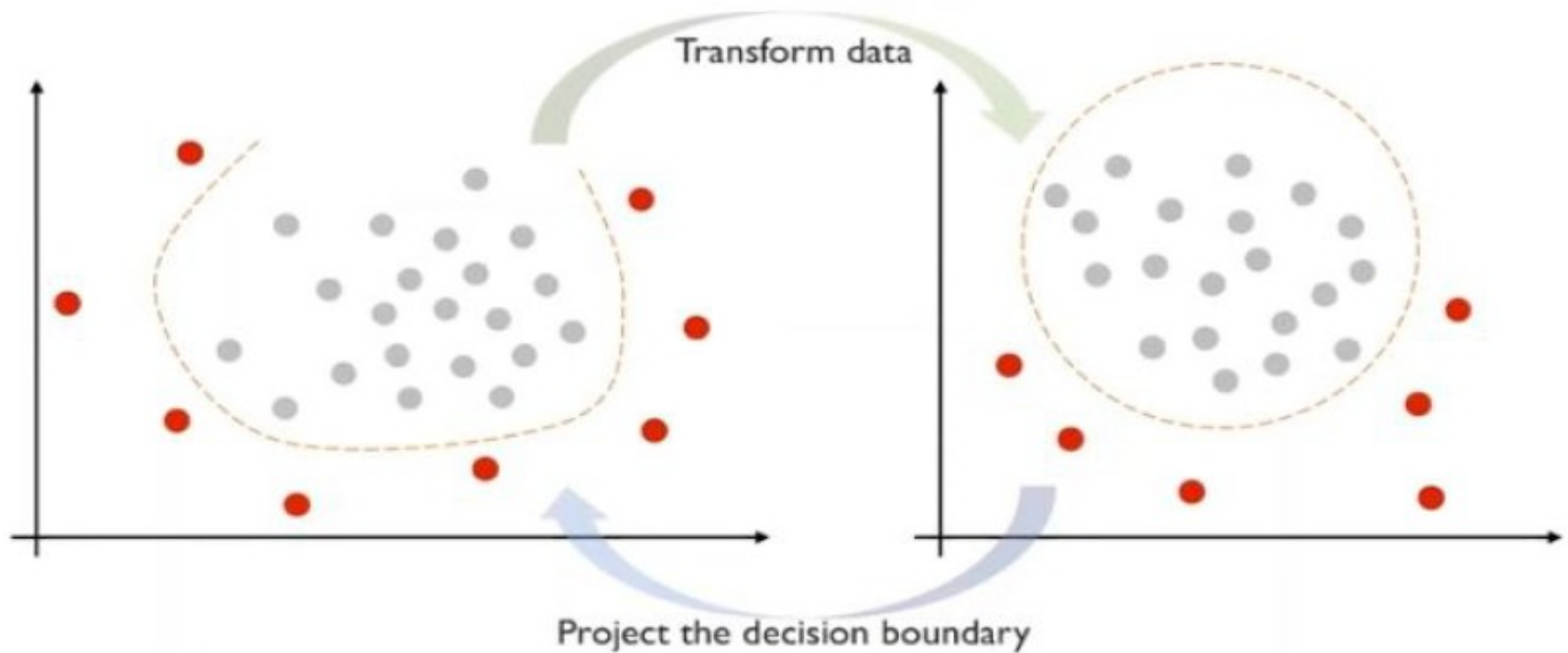
② ● $\alpha_i = C \Rightarrow \beta_i = 0 \Rightarrow \xi_i > 0$
관측치가 hypersphere 밖에 있음
(support vector)

③ ● $0 < \alpha_i < C \Rightarrow \beta_i > 0 \Rightarrow \xi_i = 0$
관측치가 hypersphere 상에 있음
(support vector)



SVDD(Suport Vector Data Description)

- 정상 관측치를 감싸는 초구를 찾아 불량 관측치를 구분하는 방법
- 초구의 경계면 바깥의 관측치들은 불량 관측치로 판단

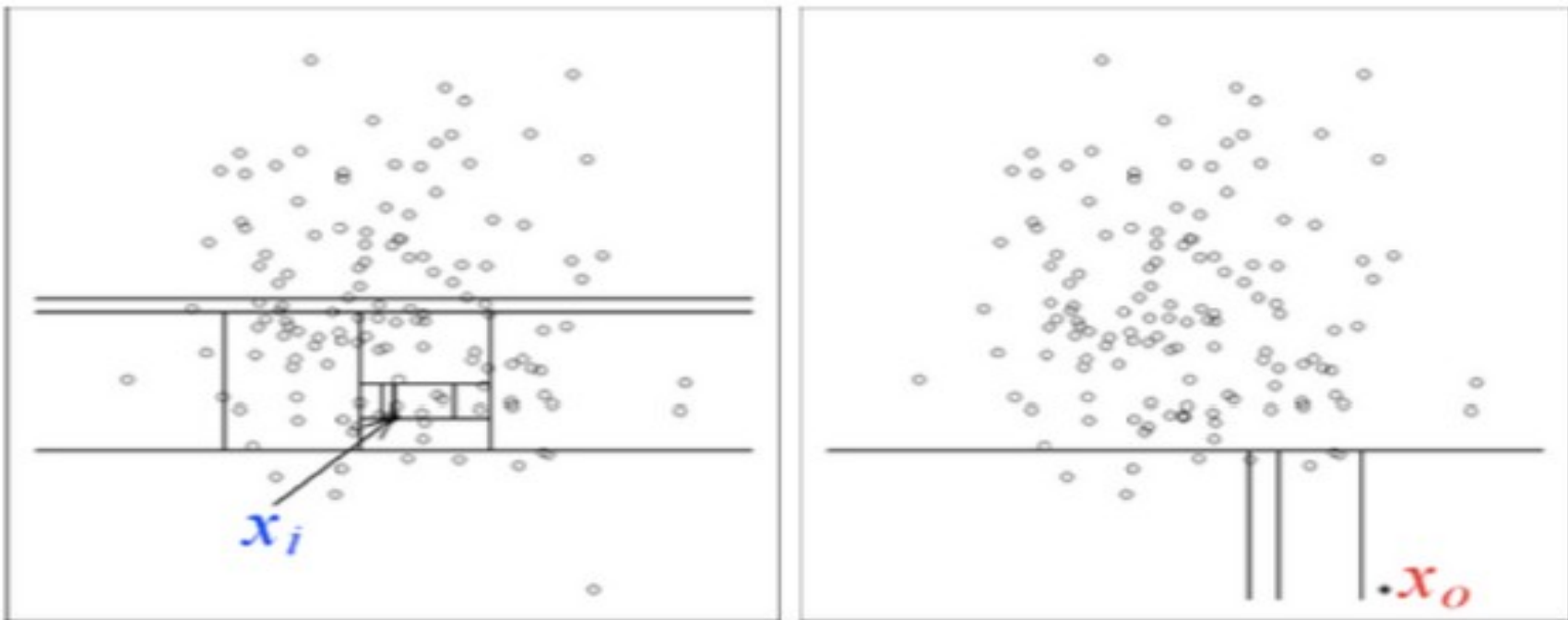


※ 구현 참조: <https://zephyrus1111.tistory.com/471#c4>

Isolation Forest

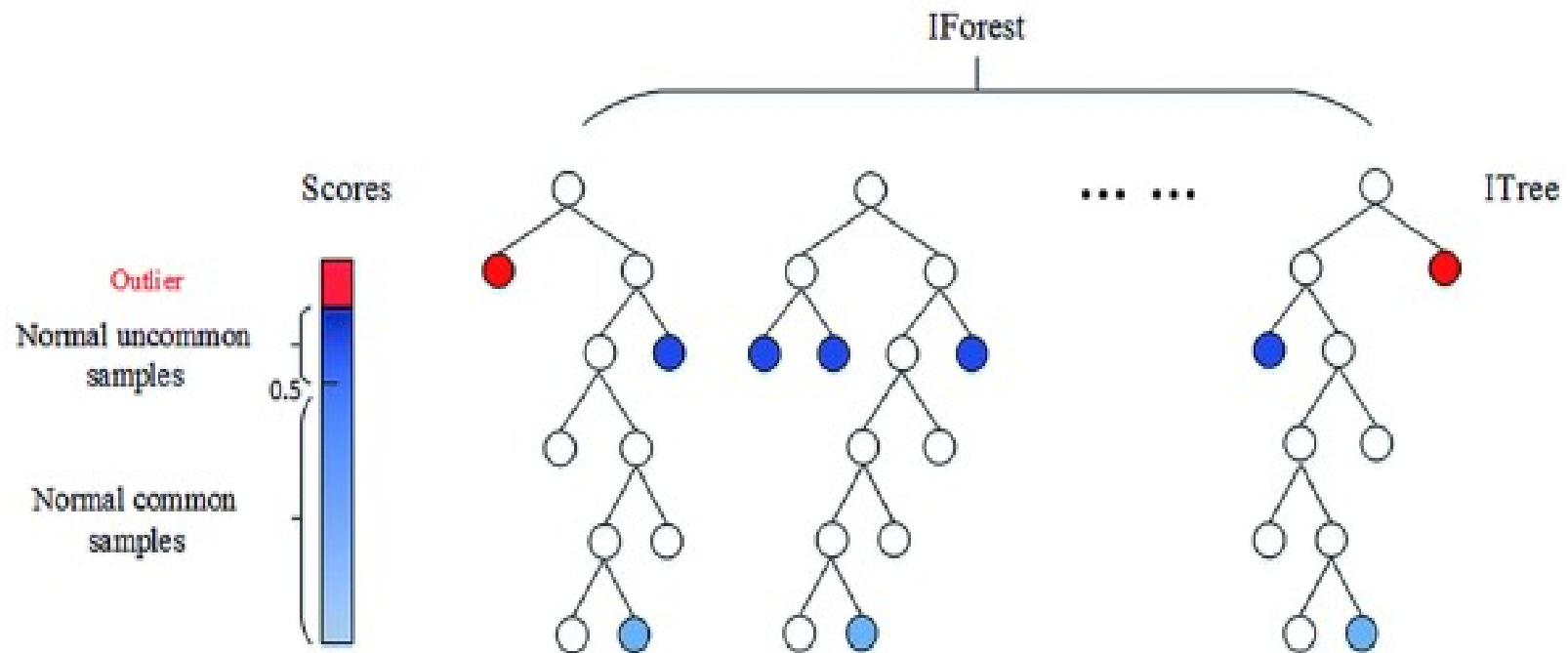
Isolation Forest

- 여기서 의사결정나무(iTree)는 각 관측치를 고립시키는 것을 목적으로 훈련된다. (이 때 레이블이 필요하지 않으므로 비지도학습이다.)
- 아래에서 보듯이 정상 관측치는 고립시키기 어렵고 이상 관측치는 고립시키기 쉽다.



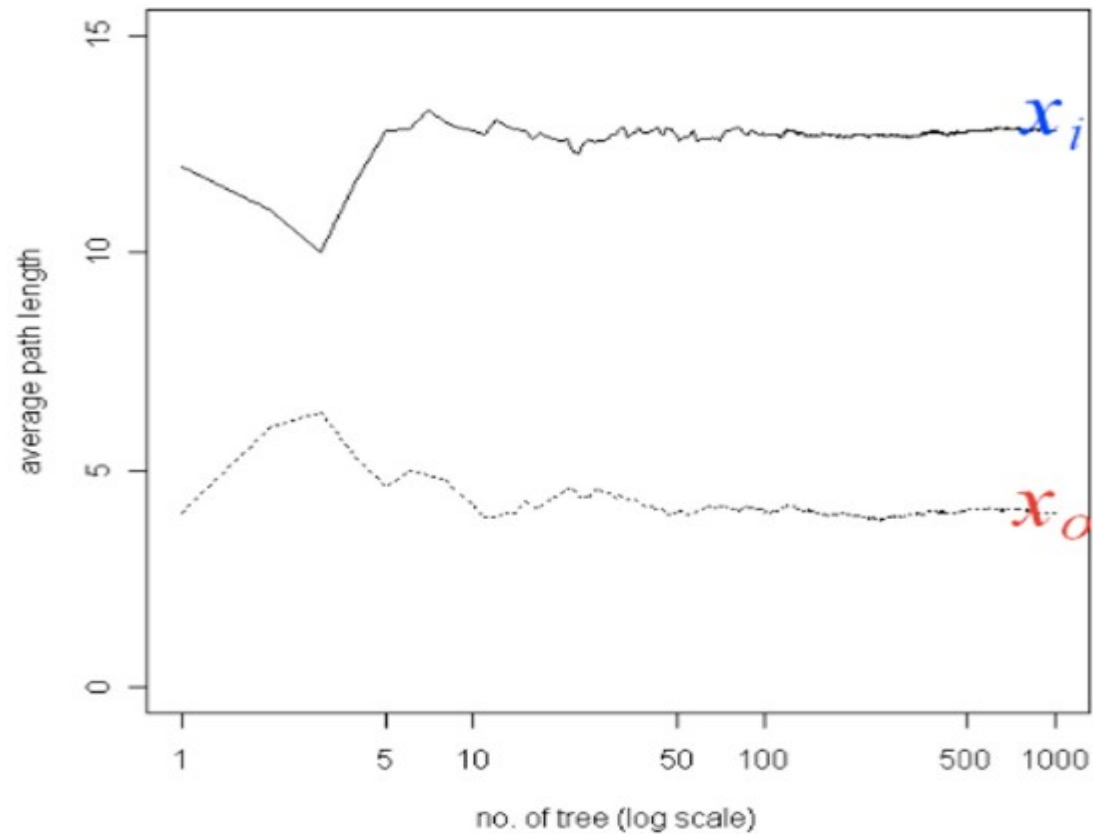
Isolation Forest

- 랜덤한 변수에 대해 랜덤한 분할(split)을 반복한다. (매번 선택한 변수의 최소값과 최대값 사이에서 근등확률로 분할점 선정)
- 이를 모든 관측치가 고립되기까지 또는 정해진 횟수 만큼 반복하고, 각 관측치가 고립될 때까지의 경로 길이(루트에서 최종 고립시키는 분할까지의 깊이)를 매번 저장한다.



Isolation Forest

- 일정 이상 반복하면 강건하게 이상치가 분리된다.



(c) Average path lengths converge

Isolation Forest

- 이상치 판단 점수

$$s(x, n) = 2^{-\frac{E(h(x))}{c(n)}}$$

- 여기서 $h(x)$ 는 각 관측치 x 의 각 iTree에서의 경로 길이
- $c(n)$: 위의 값을 정규화하기 위한 값
- 점수가 커질수록 이상치에 가깝고, 0.5정도에서 정상에 가깝다.

즉, 정상인 경우, 평균 경로길이가 평균과 유사하므로, 점수는 $1/2=0.5$ 에 가깝고, 이상인 경우는 평균길이가 짧으므로 점수는 1에 가까워진다.

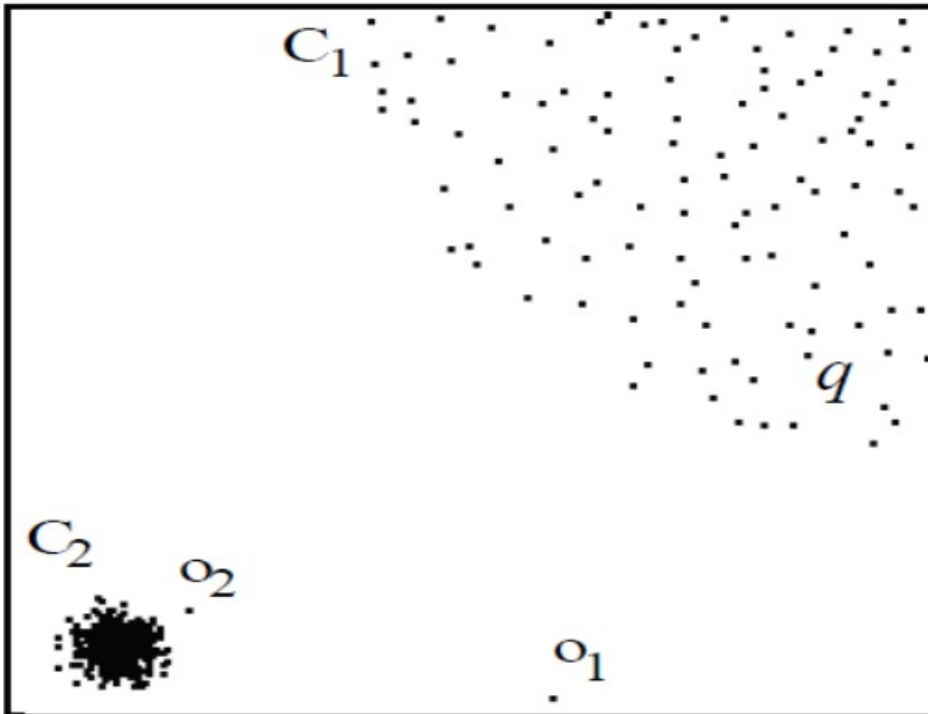
Examples

```
>>> from sklearn.ensemble import IsolationForest
>>> X = [[-1.1], [0.3], [0.5], [100]]
>>> clf = IsolationForest(random_state=0).fit(X)
>>> clf.predict([[0.1], [0], [90]])
array([ 1,  1, -1])
```

LOF

LOF(Local Outlier Factor)

- 다음 그림에서 O2는 C2 그룹에 대해 이상치인가 아닌가?

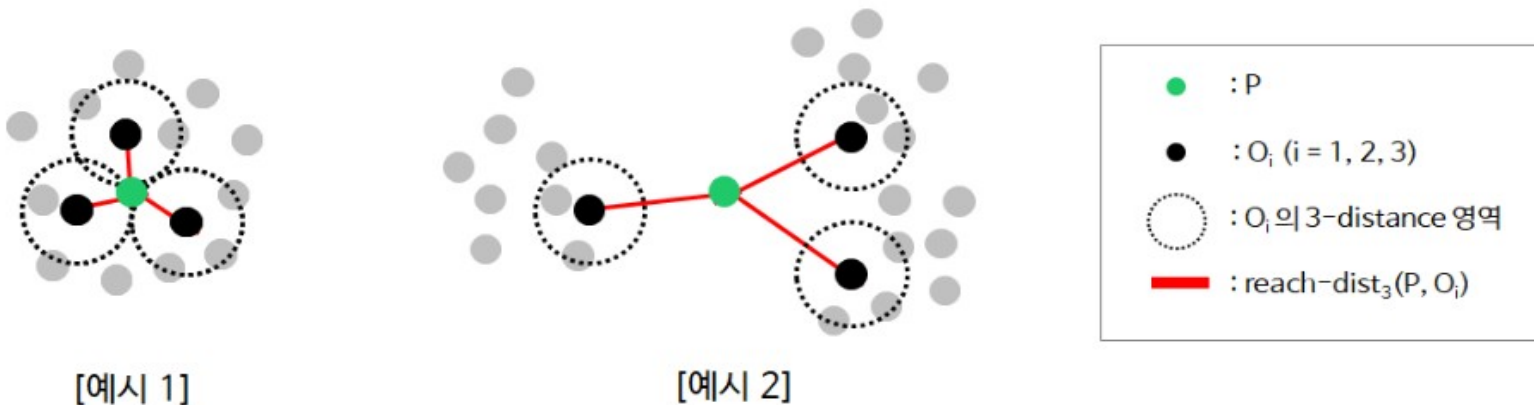


LOF(Local Outlier Factor)

- 객체 p 의 국지적 도달가능 밀도(lrd): 쉽게 표현하면 관측치 p 에 대한 k -이웃의 도달가능 거리의 평균의 역수

$$lrd_k(p) = \frac{|N_k(p)|}{\sum_{o \in N_k(p)} reach-dist_k(p, o)}$$

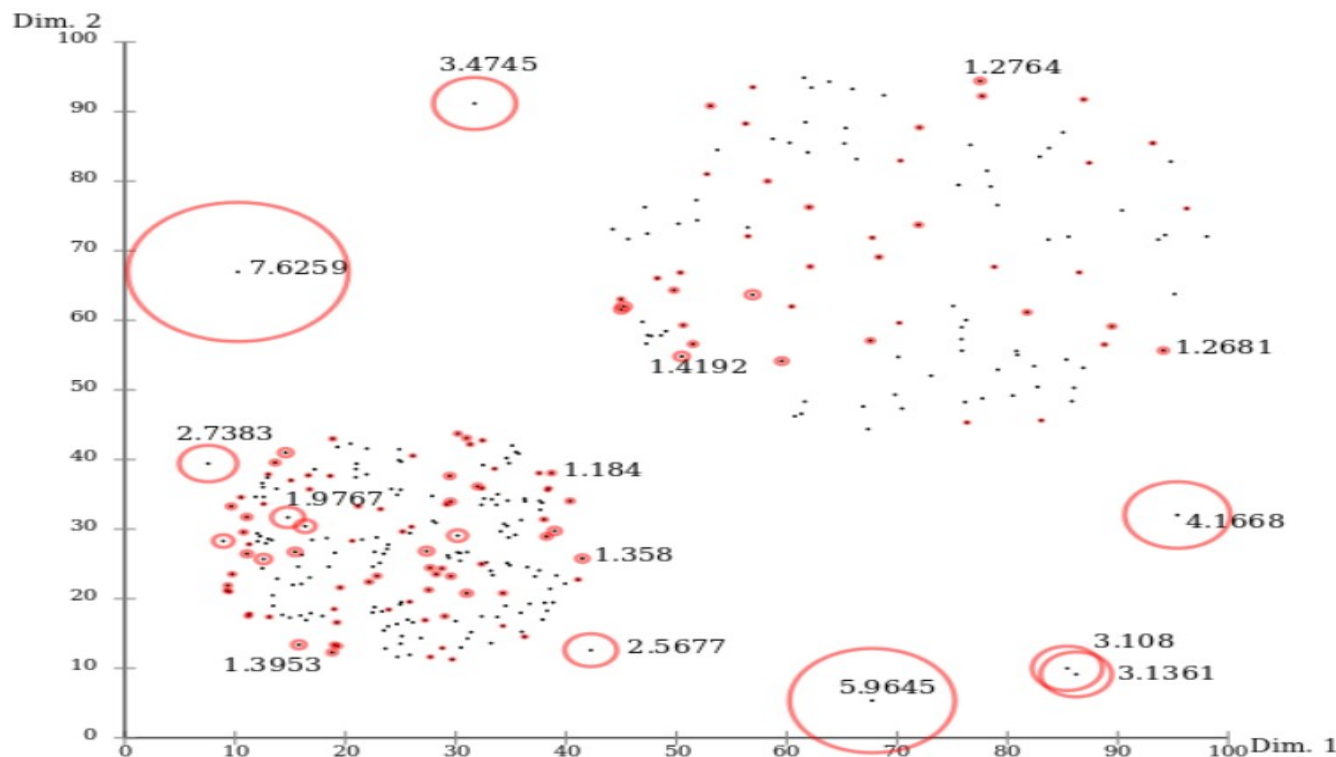
- 관측치 p 에 대한 3개의 이웃에 대한 경우



LOF(Local Outlier Factor)

- 객체 p 의 LOF: 관측치 p 의 밀도와 이웃 o 의 밀도의 비율을 평균한 것이므로, p 의 밀도가 작을수록 o 의 밀도가 클수록 LOF가 커지고 이상치라는 것이다.

$$LOF_k(p) = \frac{\sum_{o \in N_k(p)} \frac{lrd_k(o)}{lrd_k(P)}}{|N_k(P)|} = \frac{\frac{1}{lrd_k(P)} \sum_{o \in N_k(p)} lrd_k(o)}{|N_k(P)|}$$



- LOF < 1: 밀도가 높은 분포
- LOF = 1: 이웃관측치와 비슷한 분포
- LOF > 1: 밀도가 낮은 분포, 크면 이상치 정도가 큼.

LOF(Local Outlier Factor)

- 파이썬 sklearn 예제

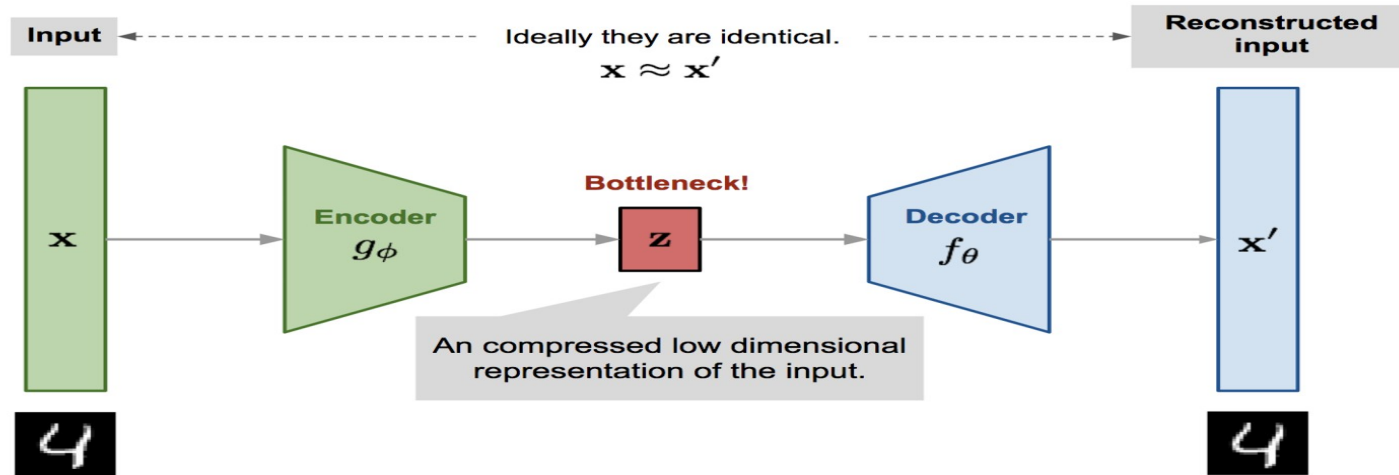
Examples

```
>>> import numpy as np
>>> from sklearn.neighbors import LocalOutlierFactor
>>> X = [[-1.1], [0.2], [101.1], [0.3]]
>>> clf = LocalOutlierFactor(n_neighbors=2)
>>> clf.fit_predict(X)
array([ 1,  1, -1,  1])
>>> clf.negative_outlier_factor_
array([-0.9821..., -1.0370..., -73.3697..., -0.9821...])
```

Autoencoder

오토인코더를 이용한 이상치 탐지

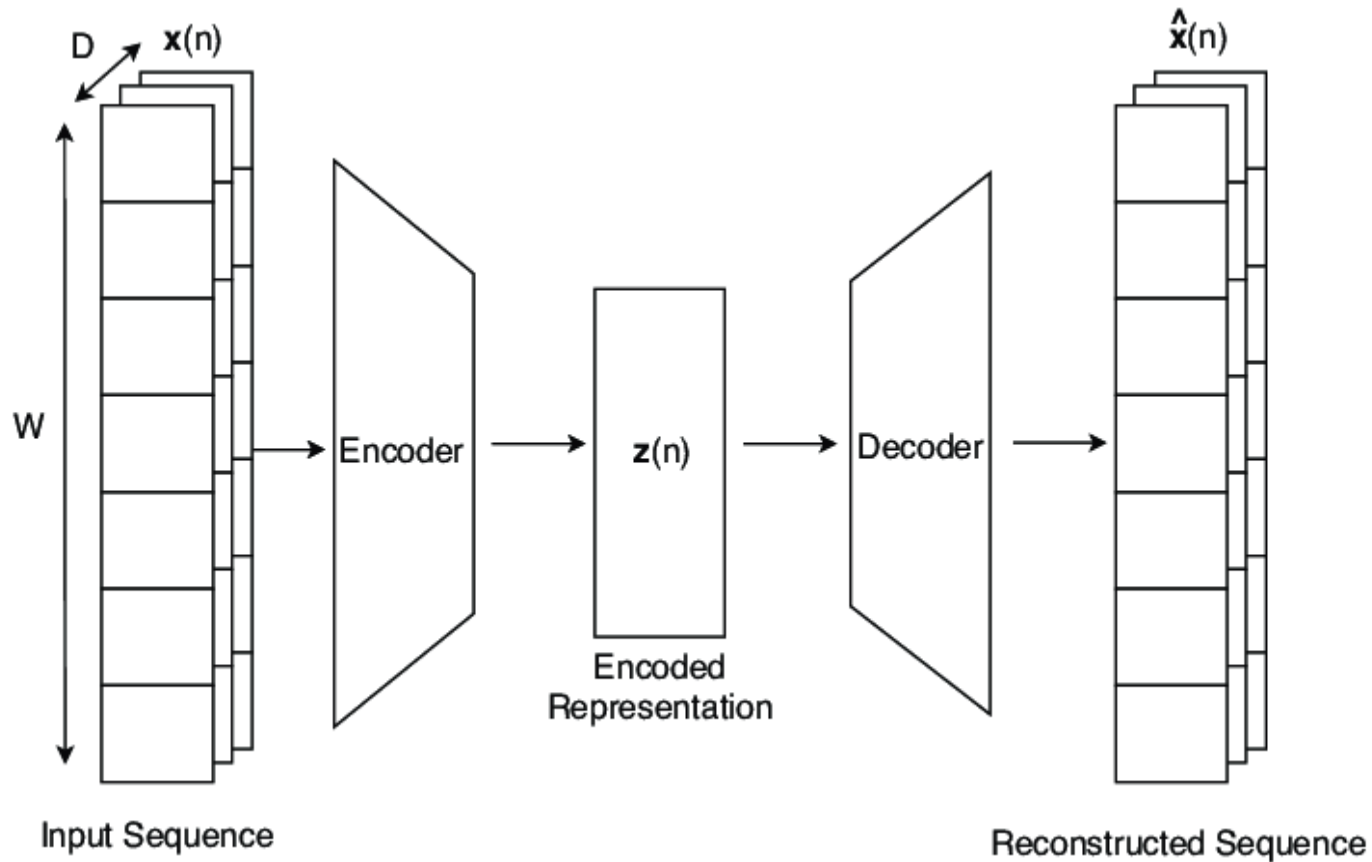
- 오토인코더(AE: Autoencoder)



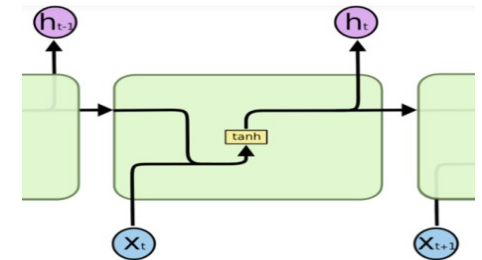
- 1) 전체 데이터를 '이상데이터'와 '정상데이터' 두 종류로 분류
- 2) 오토인코더 학습 (**※ 정상 데이터만 이용**)
- 3) 2)의 결과로 오토인코더는 정상데이터의 주요 특성을 파악하게 되므로, 잘 학습되었다면 정상적인 프로세스로부터 파생된 새로운 데이터(즉, 새로운 정상 데이터)에 대한 재구축(reconstruction)을 훌륭하게 수행
- 4) 하지만, 새로운 데이터로 이상데이터가 들어온다면 재구축이 제대로 수행되지 않을 것 (오토인코더는 학습 과정에서 이상 데이터를 다루지 않았음)
- 5) 따라서, 일정 수준 이상(cutoff)의 고 재구축 오차를 발생시키는 데이터는 이상치로 레이블링

LSTM 오토인코더를 이용한 시계열 이상치 탐지

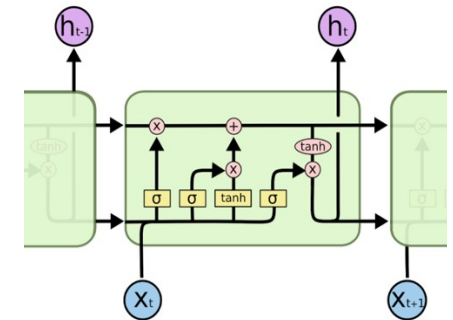
- LSTM 오토인코더: 시계열 또는 시퀀스의 재구축



- RNN



- LSTM



코드연습: https://github.com/cran2367/lstm_autoencoder_classifier/blob/master/lstm_autoencoder_classifier.ipynb